FastAPI Error-Handling Standards

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This page is how we build FastAPI handlers on the platform so that failures are predictable, observable, and never leak stack traces to callers. It applies to every request-serving service (api-service, auth-service, gateway) and to any worker-service endpoint. Two failure classes bite us repeatedly: apps that will not boot after a dependency bump, and handlers that return 500 because a coroutine was constructed but never run. Follow these standards to keep both out of production.

Standards

Pin and migrate Pydantic deliberately

Pydantic v2 changed the model API. A model that used the v1 `class Config` pattern will fail to build at import time, and the app returns 500 on startup because the ASGI application never finishes constructing. Migrate before you bump, and pin the floor.

```
# requirements.txt / pyproject
pydantic>=2.5,<3

# v1 (breaks on v2)
class Item(BaseModel):
   name: str
   class Config:
       orm_mode = True

# v2 (correct)
from pydantic import BaseModel, ConfigDict
class Item(BaseModel):
   model_config = ConfigDict(from_attributes=True)
   name: str
```

Run `bump-pydantic` across the package, then call `Model.model_rebuild()` for any model that uses forward references or string annotations, otherwise you get `PydanticUserError: ... is not fully defined`.

Always await coroutines

Calling an `async def` without `await` returns a coroutine object, not a result. Passing that object where data is expected yields a 500, and the loop logs `RuntimeWarning: coroutine 'x' was never awaited`. Make the handler `async def` and await every call. Bound every await with a timeout.

```
@app.post("/work")
async def run(req: WorkRequest) -> WorkResult:
   try:
       return await asyncio.wait_for(do_work(req), timeout=5.0)
```

```
    except asyncio.TimeoutError:
        raise HTTPException(status_code=504, detail="upstream timed out")
```

Centralize the error contract

Register one exception handler so every unhandled error returns a structured body with a stable shape and a correlation id, and log the traceback server-side only.

```
@app.exception_handler(Exception)
async def on_error(request: Request, exc: Exception):
    logger.exception("unhandled", extra={"path": request.url.path})
    return JSONResponse(status_code=500,
        content={"error": "internal_error", "trace_id": get_trace_id()})
```

- Reserve 5xx for genuine server faults; return 4xx for bad input (a 422 from schema validation is a client error, not a server bug).
- Never return raw `str(exc)` to callers.

Common failures and fixes

- App returns 500 on startup after a dependency bump; logs show `PydanticUserError: ... is not fully defined` → migrate to `model_config = ConfigDict(...)`, run `bump-pydantic`, call `model_rebuild()`, pin `pydantic>=2.5` (BUG-1001).
- POST endpoint intermittently returns 500 for valid input; loop logs `coroutine was never awaited` → add `await`, make the handler `async def`, wrap in `asyncio.wait_for` (BUG-1011).
- Callers report the API is “broken” but logs show 422 → that is client-side validation; return a field-level error body and fix the payload, do not treat it as a 5xx.

Checklist

- Pydantic pinned `>=2.5,<3`; no `class Config` remains; `model_rebuild()` called for forward refs.
- Every `async def` call site uses `await`; CI runs tests with `-W error::RuntimeWarning` to fail on un-awaited coroutines.
- Every awaited I/O call has an explicit timeout.
- A global exception handler returns a structured body with a trace id and logs the traceback server-side.
- Status codes are correct: 4xx for client/validation errors, 5xx only for server faults.
- A smoke test boots the app (import + startup event) in CI so build-time model errors fail the pipeline.

Related incidents

BUG-1001, BUG-1011